of equality propagation is routinely done by GVN. In fact, a transformation similar to this one performed by GVN was responsible for miscompiling the Rust code shown in Appendix B.

2.4 Wildcard Provenance

The previous memory model has the advantage of supporting low-level operations and enabling high-level memory optimizations. However, since integer variables now carry provenance, it makes some integer optimizations unsound. One way to solve this problem is to remove provenance from integer variables, as follows:

```
char *p = malloc(4);    // (val=0x10, obj=p)
char *q = malloc(4);    // (val=0x14, obj=q)
int v = (int)p + 4;     // (val=0x14)
int w = (int)q;         // (val=0x14)

if (v == w) {
    char *r = (int*)w;  // (val=0x14, obj=*)
    *r = 2;
}
```

The differences to the example in the previous section are that (1) integers only carry a numeric value, and (2) a pointer obtained by casting from an integer can access any object (represented with a `*`). Therefore, `v` and `w` can be used interchangeably, and GVN for integers becomes sound again. This model has a major disadvantage: precise alias analysis becomes very difficult as soon as a single integer-to-pointer cast has been performed by the program being compiled. In the next sections we explore how to recover precision.

2.5 Inbounds Pointers

In the previous section, we presented a model with wildcard provenance; this works, but it impedes precise alias analysis. In this section we explain the model currently used by LLVM where pointer arithmetic is optionally "inbounds," allowing some precision to be recovered by making out-of-bounds pointer arithmetic undefined:

```
char *p = malloc(4);  // (val=0x10, obj=p)
char *q = foo(p);     // (val=0x13, obj=p)
char *r = q +inb 2;   // poison: 0x15 is out of bounds of p

p[1] = 0;
*r = 1;               // UB
print(p[1]);          // prints 0 or 1?
```

When doing inbounds pointer arithmetic (`+inb`), the base pointer and the result must be within bounds of the same object (or one past its end). This is not the case in `r`, hence the result of the operation is **poison**, which then makes the dereference of this pointer UB [LangRef 2018].

Even if the compiler does not know the value of `q`, because of the inbounds pointer arithmetic it now knows that the minimum offset of `r` has to be two, since both `q` and `r` have to be in bounds of the same object (i.e., $0 \leq o_q \leq n$ and $0 \leq o_r \leq n$, with o_q and o_q being the offsets of `q` and `r` respectively within the object, and n the object size). Since the access to `p[1]` only accesses offset one of an object, and `*r` can only access offset two or beyond, the compiler can conclude these accesses do not alias (and that the program prints `0` always).